

Cyber Security Skill Development Lab

(2026-28)

Lab Record

Name	Archit Prakash Choudhary
Programme	M.Tech (Cybersecurity), Semester I
Roll / Registration No.	26-08-37
Section / Batch	CyberSecurity (2026-28)

Lab Assignment 2

Linux Operating System Fundamentals, Access Control and Security-Relevant Commands

1. Aim / Objective

Aim: To study the Linux operating system boot sequence, the permission and capability models, user and group management, common shells, and security-relevant commands, and to evaluate the effectiveness of Linux access-control mechanisms against the Windows mechanisms studied in Lab Assignment 1.

Objectives:

1. To understand the major stages of the Linux boot process: BIOS/UEFI → GRUB → Kernel → initramfs → systemd/init → Services/Targets → Login.
2. To understand Linux users, groups, file ownership and permissions.
3. To study the Linux read (r), write (w) and execute (x) permission model.
4. To understand special permissions such as SUID, SGID and the Sticky Bit.
5. To understand Linux capabilities and how they provide fine-grained privilege assignment.
6. To work with common Linux shells, particularly Bash and Zsh.
7. To use security-relevant Linux commands for system, user, process, file and network analysis.
8. To identify security weaknesses arising from inappropriate permissions, excessive privileges and insecure configuration.
9. To compare the Linux access-control model with the Windows security mechanisms studied in Lab Assignment 1.

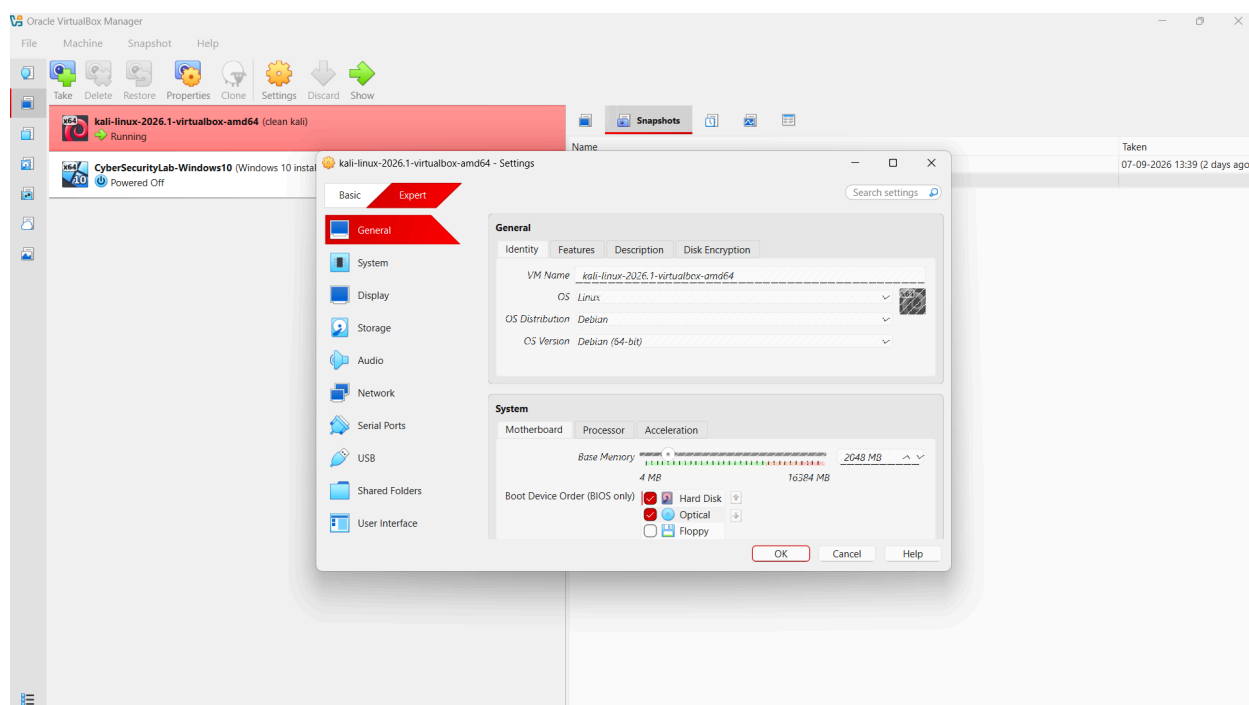
2. System / VM Configuration

The experiment was performed inside a virtual machine, as required by the assignment.

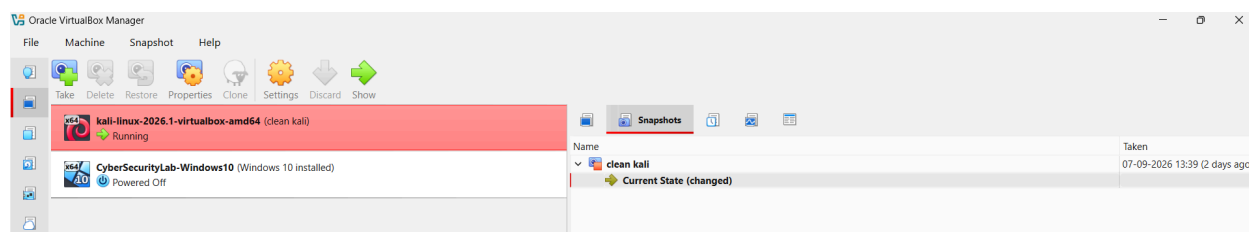
Before any security-relevant experiment was carried out, a VirtualBox snapshot of the freshly installed system was taken, so that the VM could be restored to a known-clean state if a later step (such as a permission or capability change) left the system in an inconsistent condition.

The snapshot for this lab was named "Clean Kali – Before Security Lab" and was created immediately after the base installation and first successful login, before any user, group or permission modification was made.

Parameter	Configuration
Virtualization Software	Oracle VM VirtualBox
Virtual Machine Name	kali-linux-2026.1-virtualbox-amd64
Guest Operating System	Kali GNU/Linux Rolling
Installed Version (post-update)	2026.2
Kernel	6.19.14+kali-amd64
System Architecture	x86_64 (64-bit)
Virtual Processors	2 CPU cores
Allocated RAM	4096 MB
Virtual Disk	40 GB, VDI, dynamically allocated
Network Configuration	NAT
Hostname	kali
Current / Primary User	kali
Default Shell	Zsh
Init / System Manager	systemd
Default Target	graphical.target



VirtualBox configuration for the Kali Linux VM



Snapshot taken immediately after installation, before any security experiments

3. Installation procedure

The VM was built and prepared in the following order, mirroring the approach used for the Windows VM in Lab Assignment 1:

1. Verified that Oracle VM VirtualBox was already installed and functioning correctly (the same installation used for Lab Assignment 1).
2. Created a new virtual machine, set Type to Linux and Version to Debian (64-bit), and named it kali-linux-2026.1-virtualbox-amd64.
3. Allocated 4096 MB of RAM and 2 virtual CPUs, and created a new virtual hard disk (VDI, dynamically allocated, 40 GB).
4. Attached the official Kali Linux 2026.1 installer ISO to the VM's virtual optical drive and started the VM.

5. Completed the Kali installer: selected language, keyboard layout and time zone, partitioned the virtual disk using guided partitioning, and created a normal user account named kali with a password.
6. Rebooted into the installed system, confirmed the graphical desktop loaded correctly, and then took the "Clean Kali".



Kali Linux desktop following installation

```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~$ uname -a  
Linux kali 6.19.14-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1-kali1 (2026-05-05) x86_64 GNU/Linux  
[kali@kali]~$ hostname  
kali  
[kali@kali]~$ whoami  
kali  
[kali@kali]~$ id  
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugindev),100(users),101(net dev),102(scanner),104(bluetooth),113(lpadmin),122(wireshark),123(kaboxer),124(vboxsf),125(docker)  
[kali@kali]~$ cat /etc/os-release  
PRETTY_NAME="Kali GNU/Linux Rolling"  
NAME="Kali GNU/Linux"  
VERSION_ID="2026.2"  
VERSION="2026.2"  
VERSION_CODENAME=kali-rolling  
ID=kali  
ID_LIKE=debian  
HOME_URL="https://www.kali.org/"  
SUPPORT_URL="https://forums.kali.org/"  
BUG_REPORT_URL="https://bugs.kali.org/"  
ANSI_COLOR="1;31"  
[kali@kali]~$
```

System and kernel identification for the Kali VM

Commands

Command	Interpretation	Security Relevance
<code>uname -a</code>	Displays kernel version, hostname, architecture, and other system information. The system is Kali GNU/Linux Rolling running kernel <code>6.19.14+kali-amd64</code> on <code>x86_64</code> .	Kernel information helps identify the system and determine whether the running kernel may be affected by known vulnerabilities or security advisories.
<code>hostname</code>	Displays the hostname of the machine. The observed hostname is <code>kali</code> .	Hostname can be useful for system identification and reconnaissance during security assessments.
<code>whoami</code>	Displays the username of the current effective user. The current user is <code>kali</code> .	Knowing the current user is important because the user's permissions and privileges determine what system resources and administrative operations can be accessed.
<code>id</code>	Displays the user's UID, primary GID, and supplementary group memberships. The <code>kali</code> user has UID <code>1000</code> , primary GID <code>1000</code> , and memberships including <code>sudo</code> , <code>adm</code> , <code>wireshark</code> , <code>vboxsf</code> , and <code>docker</code> .	Group membership directly affects access control. Privileged or sensitive groups such as <code>sudo</code> and <code>docker</code> can provide significant additional access, so unnecessary membership can violate the principle of least privilege.

System Identification:

The installed system identifies itself as:

`PRETTY_NAME="Kali GNU/Linux Rolling"`

```
NAME="Kali GNU/Linux"
VERSION_ID="2026.2"
VERSION="2026.2"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
```

The **ID_LIKE=debian** field indicates that Kali is based on the Debian ecosystem.

4.Part B: Linux Boot Sequence

Linux boots in a well-defined sequence that moves from firmware-level hardware initialisation through progressively higher-level software until a login prompt is reached, broadly analogous to the Windows sequence studied in Lab Assignment 1 but built around different components.

```
Power On → BIOS/UEFI → GRUB (Bootloader) → Linux Kernel → initramfs → systemd/init →  
Services/Targets → Login
```

1 BIOS/UEFI

When the virtual machine is powered on, the virtualised firmware performs the same role a physical system's firmware would: it initialises the virtual hardware, runs a basic self-test, and identifies the configured boot device. Its security importance lies in the fact that it executes before any of the operating system's own security controls are active — if the boot chain is compromised at this stage, every later control can potentially be undermined.

2 GRUB

GRUB is the bootloader responsible for locating and loading the Linux kernel. Its installed version was checked directly using command. `Grub --version`

GRUB reads its configuration, presents (or skips, on a single-OS system) a boot menu, and passes control and boot parameters to the selected kernel image. Unauthorised modification of the bootloader or its configuration is a genuine risk: an attacker with sufficient physical or VM-level access could alter which kernel is booted or inject unauthorised boot parameters, in much the same way that an attacker who can modify the Windows Boot Configuration Data (BCD) studied in Lab Assignment 1 can alter how Windows starts.

3 Linux Kernel

The kernel is the central component of the operating system: it manages CPU scheduling, memory, processes, devices, filesystems, networking and system calls, and is the layer at which most access-control decisions (permission checks, capability checks) are ultimately enforced.

4 initramfs

initramfs is a temporary, RAM-resident filesystem loaded by GRUB alongside the kernel. Its purpose is to provide just enough of an early userspace environment drivers, storage-discovery tools for the kernel to locate and mount the real root filesystem before normal operation begins.

5 systemd

This installation uses systemd as its init and service manager, confirmed by checking which process holds PID 1: /sbin/init is the traditional entry point expected by the kernel at boot; on this system it is itself a symlink into the systemd binary, so the kernel's traditional init invocation transparently starts systemd.

6 Default Target and Active Services

```
$ systemctl get-default  
graphical.target
```

graphical.target confirms that the system is configured to boot into a full graphical desktop session rather than a text-only multi-user target. The set of services systemd brings up on the way to this target was inspected next:

From a security perspective, every network-facing or privileged service that is running is part of the system's attack surface: each one is a component that must be kept patched and correctly configured. The presence of a service such as Docker or containerd here does not, by itself, indicate a vulnerability whether it represents a weakness depends on whether the service is actually required on this machine, whether it is exposed to a network, and whether it is configured correctly. This distinction is discussed further in Section 14.

7 System Status

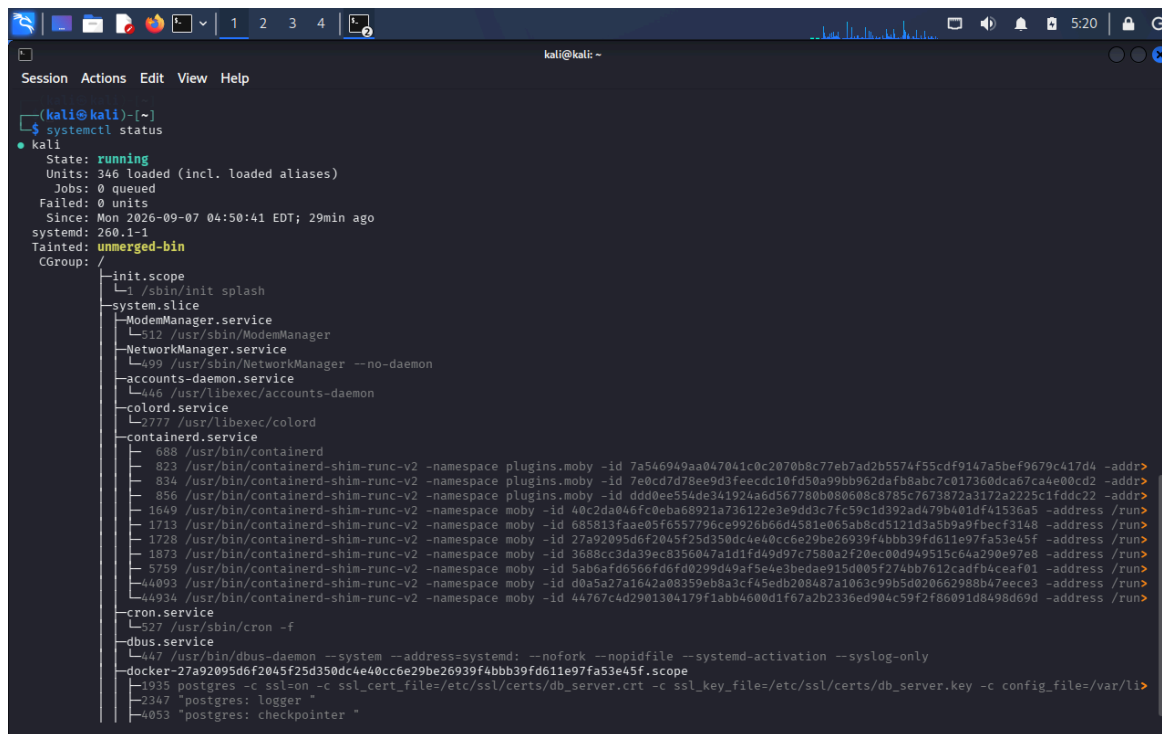
The system reported 347 loaded units with 0 failed units at the time of observation, indicating that the boot sequence completed cleanly. The status output also included a "Tainted: unmerged-bin" indicator; this is a systemd diagnostic flag related to how the filesystem's binary directories are laid out and is not, on its own, a security vulnerability

```
kali@kali: ~  
Session Actions Edit View Help  
└─(kali@kali)-[~]  
└─$ grub-install --version  
grub-install (GRUB) 2.14-2+kali1  
└─(kali@kali)-[~]  
└─$ uname -r  
6.19.14+kali-amd64  
└─(kali@kali)-[~]  
└─$ systemctl get-default  
graphical.target  
└─(kali@kali)-[~]  
└─$ ps -p 1 -o pid,comm,args  
PID COMMAND  
1 systemd /sbin/init splash  
└─(kali@kali)-[~]  
└─$  
└─(kali@kali)-[~]  
└─$  
└─(kali@kali)-[~]  
└─$ systemctl list-units --type=service  
UNIT LOAD ACTIVE SUB DESCRIPTION  
accounts-daemon.service loaded active running Accounts Service  
colord.service loaded active running Manage, Install and Generate Color Profiles  
console-setup.service loaded active exited Set console font and keymap  
containerd.service loaded active running containerd container runtime  
cron.service loaded active running Regular background program processing daemon  
dbus.service loaded active running D-Bus System Message Bus  
docker.service loaded active running Docker Application Container Engine  
getty@tty1.service loaded active running Getty on tty1  
haveged.service loaded active running Entropy Daemon based on the HAVEGE algorithm  
ifupdown-pre.service loaded active exited Helper to synchronize boot up for ifupdown  
kasm-network-plugin.service loaded active exited Kasm Network plugin for Docker  
kasm.service loaded active exited Kasm Workspaces Stack  
keyboard-setup.service loaded active exited Set the console keyboard layout  
kmod-static-nodes.service loaded active exited Create List of Static Device Nodes  
lightdm.service loaded active running Light Display Manager  
ModemManager.service loaded active running Modem Manager  
networking.service loaded active exited Raise network interfaces
```

Kernel release, default systemd target, PID 1, and the start of systemctl status output.

```
kali@kali: ~  
Session Actions Edit View Help  
└─(kali@kali)-[~]  
└─$ systemctl list-units --type=service  
UNIT LOAD ACTIVE SUB DESCRIPTION  
accounts-daemon.service loaded active running Accounts Service  
colord.service loaded active running Manage, Install and Generate Color Profiles  
console-setup.service loaded active exited Set console font and keymap  
containerd.service loaded active running containerd container runtime  
cron.service loaded active running Regular background program processing daemon  
dbus.service loaded active running D-Bus System Message Bus  
docker.service loaded active running Docker Application Container Engine  
getty@tty1.service loaded active running Getty on tty1  
haveged.service loaded active running Entropy Daemon based on the HAVEGE algorithm  
ifupdown-pre.service loaded active exited Helper to synchronize boot up for ifupdown  
kasm-network-plugin.service loaded active exited Kasm Network plugin for Docker  
kasm.service loaded active exited Kasm Workspaces Stack  
keyboard-setup.service loaded active exited Set the console keyboard layout  
kmod-static-nodes.service loaded active exited Create List of Static Device Nodes  
lightdm.service loaded active running Light Display Manager  
ModemManager.service loaded active running Modem Manager  
networking.service loaded active exited Raise network interfaces  
NetworkManager-wait-online.service loaded active exited Network Manager Wait Online  
NetworkManager.service loaded active running Network Manager  
plymouth-quit-wait.service loaded active exited Hold until boot process finishes up  
plymouth-read-write.service loaded active exited Tell Plymouth To Write Out Runtime Data  
plymouth-start.service loaded active exited Show Plymouth Boot Screen  
polkit.service loaded active running Authorization Manager  
rpc-statd-notify.service loaded active exited Notify NFS peers of a restart  
rtkit-daemon.service loaded active running RealtimeKit Scheduling Policy Service  
systemd-binfmt.service loaded active exited Set Up Additional Binary Formats  
systemd-journal-flush.service loaded active exited Flush Journal to Persistent Storage  
systemd-journald.service loaded active running Journal Service  
systemd-logind.service loaded active running User Login Management  
systemd-modules-load.service loaded active exited Load Kernel Modules  
systemd-random-seed.service loaded active exited Load/Save OS Random Seed  
systemd-remount-fs.service loaded active exited Remount Root and Kernel File Systems  
systemd-sysctl.service loaded active exited Apply Kernel Variables  
systemd-tmpfiles-setup-dev-early.service loaded active exited Create Static Device Nodes in /dev gracefully  
systemd-tmpfiles-setup-dev.service loaded active exited Create Static Device Nodes in /dev  
systemd-tmpfiles-setup.service loaded active exited Create System Files and Directories  
systemd-udev-load-credentials.service loaded active exited Load udev Rules from Credentials  
systemd-udev-trigger.service loaded active exited Coldplug All udev Devices  
systemd-udevvd.service loaded active running Rule-based Manager for Device Events and Files
```

Running services

A terminal window titled 'kali@kali' showing the output of the 'systemctl status' command. The output indicates the system is 'running' with 346 loaded units. It lists various services like 'init.scope', 'system.slice', 'ModemManager.service', 'NetworkManager.service', 'accounts-daemon.service', 'colord.service', 'containerd.service', 'cron.service', and 'dbus.service'. The 'containerd.service' section is expanded, showing a list of container instances with their IDs and addresses. The terminal has a dark background with light-colored text.

Terminal output of systemctl status

Q: What could happen if the bootloader is modified?

An attacker who can modify GRUB or its configuration can change which kernel or operating system is loaded, or inject unauthorised kernel parameters. This could be used to boot a modified kernel, disable security-relevant kernel options, or boot into a maintenance/rescue mode that bypasses normal authentication entirely.

Q: Why is boot integrity important?

Boot integrity matters because the boot process runs before almost all of the operating system's normal security controls are active. If the boot chain firmware, bootloader, kernel is compromised, everything that loads afterwards, including the access-control mechanisms this report studies, can no longer be fully trusted, since a compromised kernel can lie to every layer above it.

Q: What security risks arise from unauthorised modification of boot parameters?

Kernel boot parameters can disable security features (for example, memory-protection or module-signing options), change which init system or target is used, or in some configurations grant a root shell directly at boot without authentication. Restricting who can edit the GRUB configuration and boot menu is therefore itself a security control.

Q: How does the Linux boot process differ from the Windows boot process studied in Lab Assignment 1?

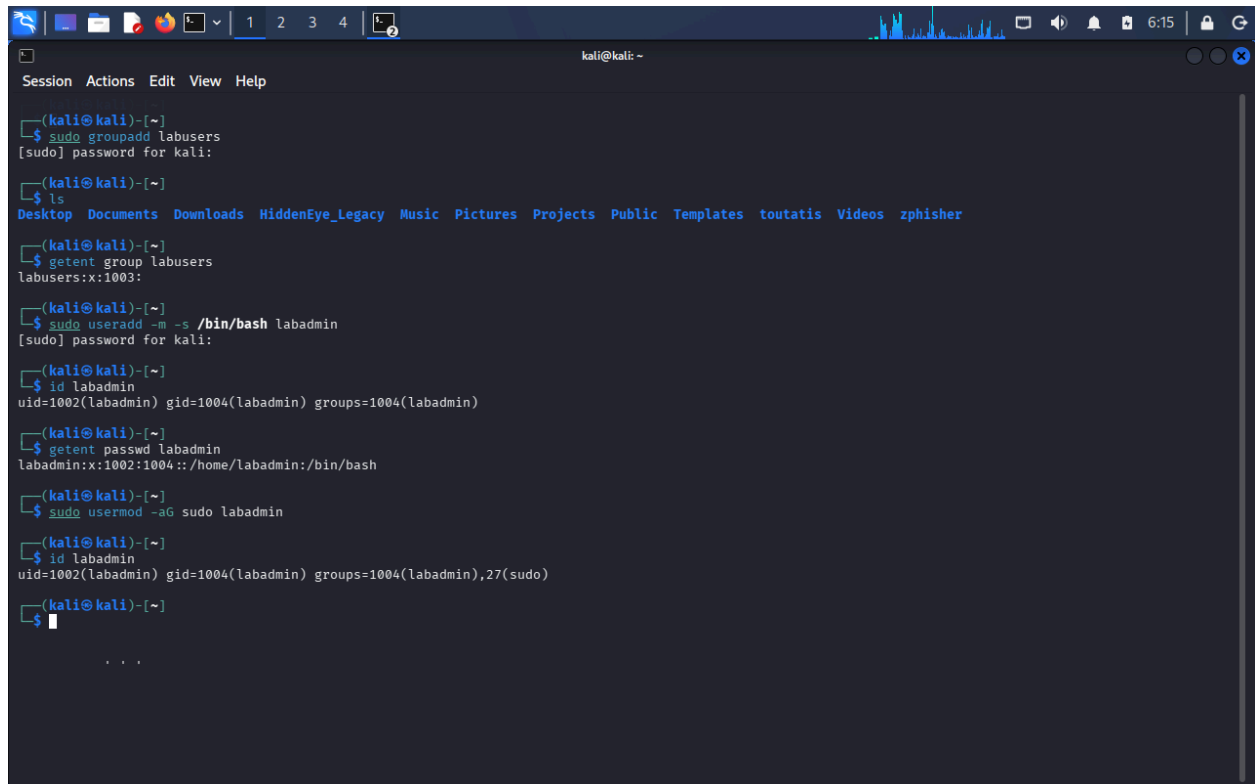
Both operating systems follow the same broad pattern of firmware, followed by a dedicated boot-management component, followed by kernel initialisation and then service start-up. Windows uses the Windows Boot Manager together with the Boot Configuration Data (BCD) store and winload.efi, protected where enabled by Secure Boot; Linux, on this system, uses GRUB reading its own configuration and handing control to the Linux kernel. Where Windows brings up the Service Control Manager and Windows Services, Linux brings up systemd and its unit files. The underlying goal reach a known, service-ready state before allowing a user login is the same in both cases, but the specific trusted components differ, and each has its own equivalent integrity concern: an unauthorised change to the BCD on Windows is conceptually the same class of risk as an unauthorised change to the GRUB configuration on Linux.

5. PartC: Linux Users and Groups

Linux uses numeric user and group identifiers, together with group membership, as the fundamental building blocks of its access-control model the same role that user accounts and Security Identifiers (SIDs) played for Windows in Lab Assignment 1. To exercise this model directly, one administrative-capable user, two ordinary users and one dedicated group were created inside the VM.

User	UID	Primary GID	Privilege / Group	Purpose
kali	1000	1000	sudo + supplementary groups	Default account created at installation
labadmin	1002	1004	sudo	Administrative-capable account for this lab
labuser1	1003	1005	labusers	Ordinary user, member of the lab group
labuser2	1004	1006	labusers	Ordinary user, member of the lab group

The dedicated laboratory group, labusers, was created with GID 1003 and used to demonstrate group-based rather than per-user permission management.



```
(kali@kali)-[~]
$ sudo groupadd labusers
[sudo] password for kali:

(kali@kali)-[~]
$ ls
Desktop  Documents  Downloads  HiddenEye_Legacy  Music  Pictures  Projects  Public  Templates  toutatis  Videos  zphisher

(kali@kali)-[~]
$ getent group labusers
labusers:x:1003:

(kali@kali)-[~]
$ sudo useradd -m -s /bin/bash labadmin
[sudo] password for kali:

(kali@kali)-[~]
$ id labadmin
uid=1002(labadmin) gid=1004(labadmin) groups=1004(labadmin)

(kali@kali)-[~]
$ getent passwd labadmin
labadmin:x:1002:1004::/home/labadmin:/bin/bash

(kali@kali)-[~]
$ sudo usermod -aG sudo labadmin

(kali@kali)-[~]
$ id labadmin
uid=1002(labadmin) gid=1004(labadmin) groups=1004(labadmin),27(sudo)

(kali@kali)-[~]
$
```

Creation and verification of the labusers group and then labadmin created with a home directory, then granted sudo membership

useradd -m -s /bin/bash created labadmin with a home directory (/home/labadmin) and Bash as its login shell. At creation the account had no administrative capability at all; it was only after the explicit usermod -aG sudo labadmin step that labadmin's group list gained the sudo entry, which is the specific group that grants sudo access on this Debian-derived system.

```
kali@kali: ~  
Session Actions Edit View Help  
uid=1002(labadmin) gid=1004(labadmin) groups=1004(labadmin)  
  
(kali@kali)-[~]  
$ getent passwd labadmin  
labadmin:x:1002:1004::/home/labadmin:/bin/bash  
  
(kali@kali)-[~]  
$ sudo usermod -aG sudo labadmin  
  
(kali@kali)-[~]  
$ id labadmin  
uid=1002(labadmin) gid=1004(labadmin) groups=1004(labadmin),27(sudo)  
  
(kali@kali)-[~]  
$ getent group sudo  
sudo:x:27:kali,labadmin  
  
(kali@kali)-[~]  
$ sudo useradd -m -s /bin/bash labuser1  
  
(kali@kali)-[~]  
$ sudo useradd -m -s /bin/bash labuser2  
  
(kali@kali)-[~]  
$ id labuser1 labuser2  
uid=1003(labuser1) gid=1005(labuser1) groups=1005(labuser1)  
uid=1004(labuser2) gid=1006(labuser2) groups=1006(labuser2)  
  
(kali@kali)-[~]  
$ getent group sudo  
sudo:x:27:kali,labadmin  
  
(kali@kali)-[~]  
$ sudo passwd labuser1  
New password:  
Retype new password:  
passwd: password updated successfully  
  
(kali@kali)-[~]  
$ sudo passwd labuser2  
New password:  
Retype new password:  
passwd: password updated successfully
```

labuser1 and labuser2 shown as supplementary members of the labusers group

```
$ getent passwd labadmin
```

Output - labadmin:x:1002:1004::/home/labadmin:/bin/bash

```
$ getent passwd labadmin
```

Output - labadmin:x:1002:1004::/home/labadmin:/bin/bash

The colon-separated fields represent, in order: username, a placeholder in place of the password (the letter x, since the encrypted password itself is kept in the separately protected /etc/shadow file, not in the world-readable /etc/passwd), UID, GID, the GECOS comment field, the home directory, and the login shell.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ getent passwd labuser1  
labuser1:x:1003:1005::/home/labuser1:/bin/bash  
  
(kali@kali)-[~]  
$ getent group labusers  
labusers:x:1003:labuser1,labuser2  
  
(kali@kali)-[~]  
$ groups labuser1  
labuser1 : labuser1 labusers
```

Getent command

Q1. What is a UID?

A UID (User Identifier) is the numerical value the kernel actually uses to identify a Linux account; usernames are only a human-readable label mapped to this number in `/etc/passwd`. In this lab, the kali account carries UID 1000.

Q2. What is the significance of UID 0?

UID 0 is reserved for the root, or superuser, identity. A process running with UID 0 is not subject to the normal file-permission checks and can perform almost any operation on the system, which is why UID 0 is treated as the single most security-critical identifier on a Linux system.

Q3. Why should routine activities not be performed as root?

Operating routinely as root violates the principle of least privilege: every command run, every file opened, and every script executed inherits root's unrestricted authority, so a mistake or a compromised program has system-wide consequences. A normal, unprivileged account should be used for everyday work, with root access invoked only for the specific operation that actually requires it the same reasoning that motivates using a standard Windows account rather than staying logged in as Administrator, as discussed in Lab Assignment 1.

Q4. What is the difference between a primary and a supplementary group?

A user's primary group is the group recorded against their account in `/etc/passwd` and is the group new files they create are normally assigned to. Supplementary groups are additional memberships listed in `/etc/group` that extend a user's access without changing their primary group labuser1 and labuser2, for example, each keep their own primary group but gain access to labusers-owned resources purely through supplementary membership.

Q5. How does Linux user/group management compare with Windows users and groups?

Both systems separate an internal numeric or structured identifier from the human-readable account name: Linux uses a UID together with a GID, while Windows generates a Security Identifier (SID) for every account and group. Both also support supplementary group membership, so that access can be granted collectively rather than one account at a time. Linux's labusers group here plays exactly the role that the CyberLabUsers local group played for Student1 and Student2 in Lab Assignment 1. The main structural difference is that Windows access control is normally expressed through discretionary ACLs attached to each object, listing SIDs and their permitted operations, whereas traditional Linux permissions instead attach a single, fixed owner/group/others triple directly to each file, with POSIX ACLs available as an optional extension where finer control is required.

6. Part D :Linux File Permission Model

Linux represents file access using three categories -Owner, Group and Others each of which can independently be granted read (r), write (w) and execute (x) permission. The assignment specifically called for experimenting with `chmod`, `chown` and `chgrp` across different users, using a dedicated laboratory directory

```
(kali@kali)-[~]
$ mkdir ~/CyberSecurityLab

(kali@kali)-[~]
$ ls -ld ~/CyberSecurityLab
drwxrwxr-x 2 kali kali 4096 Sep  7 06:36 /home/kali/CyberSecurityLab

(kali@kali)-[~]
$ touch ~/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ /home/kali/CyberSecurityLab/testfile.txt
bash: /home/kali/CyberSecurityLab/testfile.txt: Permission denied
```

*Created the Laboratory Directory and Test File Using mkdir and touch command
And checking permission using ls command*

Understanding Permission Values

Linux numeric permissions use:

Permission	Numeric value
Read (r)	4
Write (w)	2
Execute (x)	1

Therefore:

$rwx = 4 + 2 + 1 = 7$; $rw- = 4 + 2 = 6$; $r-x = 4 + 1 = 5$; $r-- = 4$

For example:

640

means:

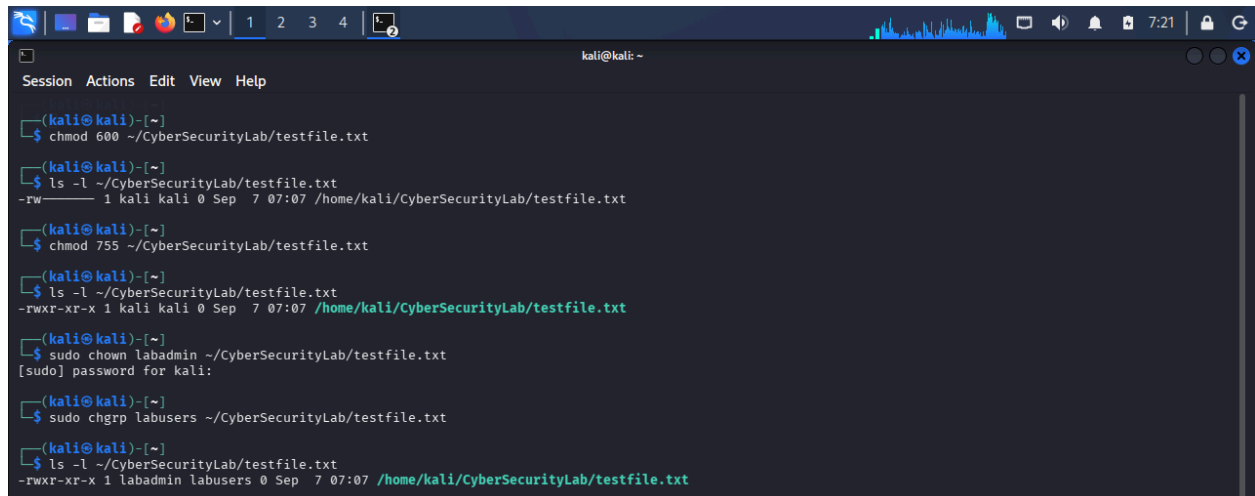
Owner = 6 = rw-

Group = 4 = r--

Others = 0 = —

chmod 600 and chmod 755

600 leaves only the owner able to read or write the file, and is appropriate for anything that should be visible to no one else on the system, such as a private key or a configuration file containing credentials.

A terminal window on a Kali Linux system. The window title is 'kali@kali: ~'. The menu bar shows 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a series of commands and their outputs: 1. 'chmod 600 ~/CyberSecurityLab/testfile.txt' is executed. 2. 'ls -l ~/CyberSecurityLab/testfile.txt' shows '-rw----- 1 kali kali 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt'. 3. 'chmod 755 ~/CyberSecurityLab/testfile.txt' is executed. 4. 'ls -l ~/CyberSecurityLab/testfile.txt' shows '-rwxr-xr-x 1 kali kali 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt'. 5. 'sudo chown labadmin ~/CyberSecurityLab/testfile.txt' is executed, followed by the prompt '[sudo] password for kali:'. 6. 'sudo chgrp labusers ~/CyberSecurityLab/testfile.txt' is executed. 7. 'ls -l ~/CyberSecurityLab/testfile.txt' shows '-rwxr-xr-x 1 labadmin labusers 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt'.

```
(kali@kali)-[~]
$ chmod 600 ~/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ ls -l ~/CyberSecurityLab/testfile.txt
-rw----- 1 kali kali 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ chmod 755 ~/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ ls -l ~/CyberSecurityLab/testfile.txt
-rwxr-xr-x 1 kali kali 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ sudo chown labadmin ~/CyberSecurityLab/testfile.txt
[sudo] password for kali:

(kali@kali)-[~]
$ sudo chgrp labusers ~/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ ls -l ~/CyberSecurityLab/testfile.txt
-rwxr-xr-x 1 labadmin labusers 0 Sep 7 07:07 /home/kali/CyberSecurityLab/testfile.txt
```

755 grants the owner full read/write/execute access while everyone else can read and execute but not modify the file a common setting for scripts and directories that should be publicly usable but not publicly editable, though it should not be applied automatically to every resource without considering whether public read/execute access is actually intended.

Changing Ownership

The file owner was changed using:

```
sudo chown labadmin ~/CyberSecurityLab/testfile.txt
```

The group was changed using:

```
sudo chgrp labusers ~/CyberSecurityLab/testfile.txt
```

The resulting ownership was:

Owner → labadmin

Group → labusers

The final permission was set to: `sudo chmod 640 ~/CyberSecurityLab/testfile.txt`

Verification: `stat -c '%A %a %U %G %n' ~/CyberSecurityLab/testfile.txt`

Observed: `-rw-r----- 640 labadmin labusers /home/kali/CyberSecurityLab/testfile.txt`

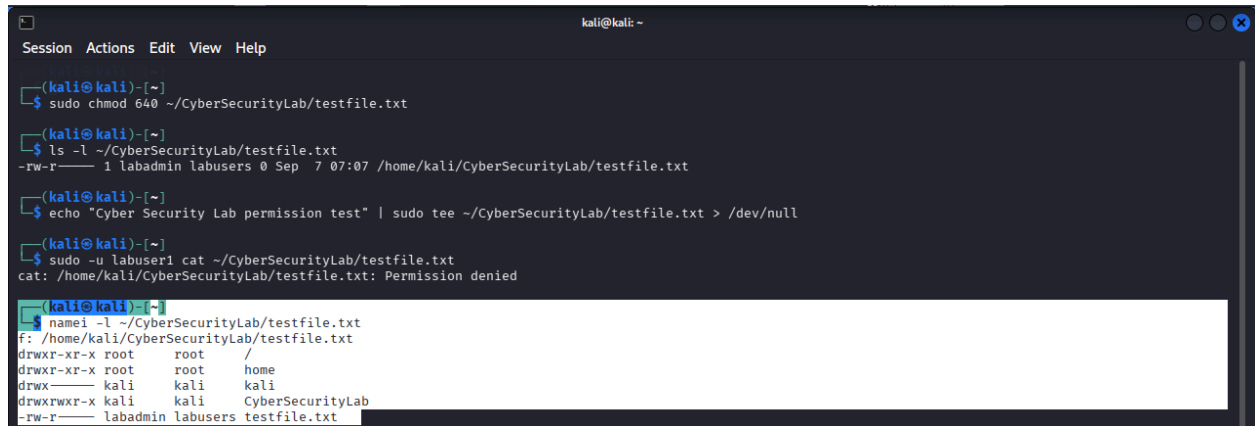
Permission Test with labuser1

This result is initially counter-intuitive, since labuser1 is a member of the labusers group and the file's group permission (r--) should allow it to read the file. The cause was traced using `namei`, which resolves and prints the permissions on every directory in the path:

The important observation is that `/home/kali` is:

`drwx-----`

Therefore `labuser1` cannot traverse `/home/kali`, even though the file itself grants read permission to the `labusers` group.

A terminal window titled 'kali@kali ~' showing a series of commands and their outputs. The commands include setting permissions on a file, listing the file, writing to it, and attempting to read it as a different user. The final command, 'namei -l ~/CyberSecurityLab/testfile.txt', produces a detailed output showing the directory traversal path and the permissions at each step, highlighting that the '/home/kali' directory blocks traversal for other users.

```
(kali@kali)~$ sudo chmod 640 ~/CyberSecurityLab/testfile.txt
(kali@kali)~$ ls -l ~/CyberSecurityLab/testfile.txt
-rw-r----- 1 labadmin labusers 0 Sep  7 07:07 /home/kali/CyberSecurityLab/testfile.txt
(kali@kali)~$ echo "Cyber Security Lab permission test" | sudo tee ~/CyberSecurityLab/testfile.txt > /dev/null
(kali@kali)~$ sudo -u labuser1 cat ~/CyberSecurityLab/testfile.txt
cat: /home/kali/CyberSecurityLab/testfile.txt: Permission denied
(kali@kali)~$ namei -l ~/CyberSecurityLab/testfile.txt
f: /home/kali/CyberSecurityLab/testfile.txt
drwxr-xr-x root root /
drwxr-xr-x root root home
drwx----- kali kali kali
drwxrwxr-x kali kali CyberSecurityLab
-rw-r----- labadmin labusers testfile.txt
```

namei -l revealing that `/home/kali` itself blocks traversal for other users

Security concept:

File permissions alone are not sufficient. A user must also have appropriate execute/traverse permission on the directories forming the path. This is an important practical distinction between file access and directory access.

Permission Matrix

Based on the final file configuration:

Resource: `testfile.txt` , Owner = labadmin , Group = labusers , Permission = 640

User	Resource	Read	Write	Execute	Result
labadmin	testfile.txt	Yes	Yes	No	Full read/write access (owner)
labuser1	testfile.txt	Yes*	No	No	Denied in practice blocked by <code>/home/kali</code> traversal

labuser2	testfile.txt	Yes*	No	No	Denied in practice blocked by /home/kali traversal
kali (other)	testfile.txt	No	No	No	No access (others)
labuser1	~/CyberSecurityLab	Yes	Yes	Yes	Directory itself is 775, reachable if parent allowed it

* The file-level permission grants read access to the **labusers** group, but in this experiment **labuser1** could not reach the file because **/home/kali** denied directory traversal.

This demonstrates why access control must be evaluated across the entire filesystem path.

7. Part E : Numeric and Symbolic Permissions

Linux permissions can be expressed symbolically (u+r, g-w, o=rx) or numerically, where read, write and execute are assigned the values 4, 2 and 1 respectively and summed per category:

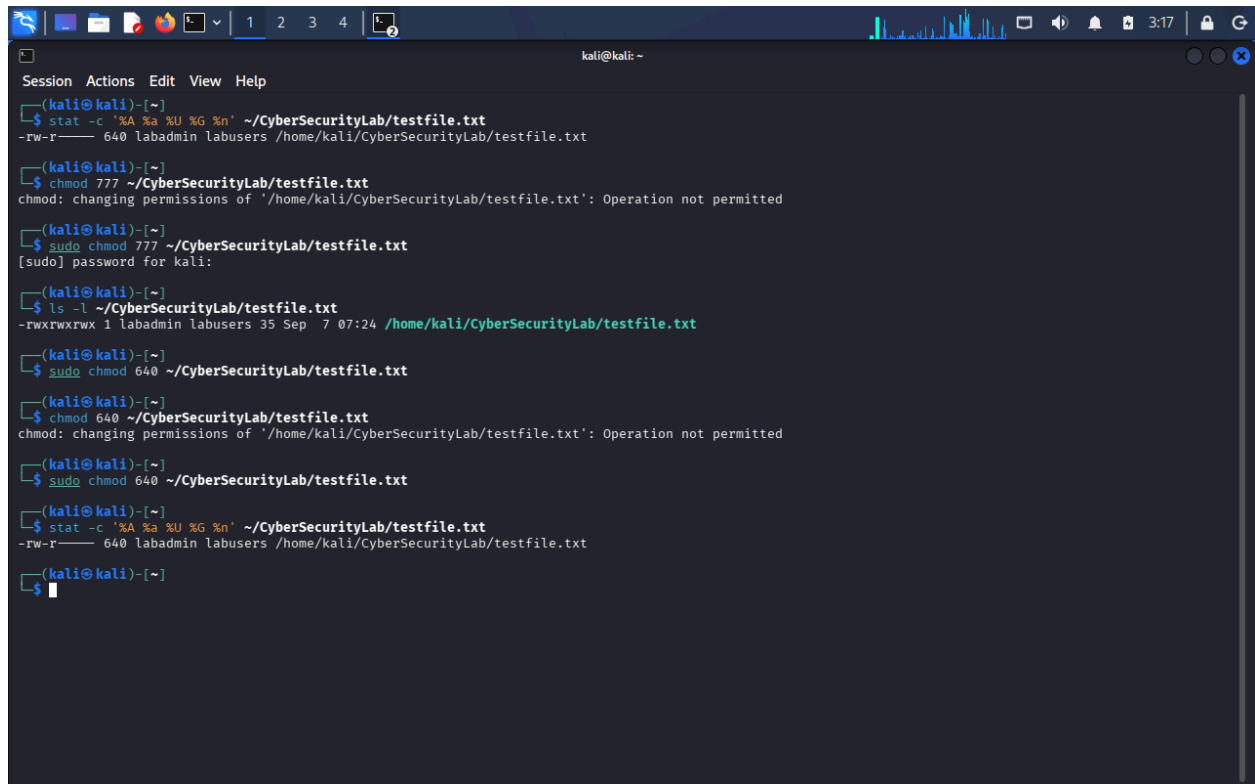
$rw\text{x} = 4 + 2 + 1 = 7$ $r\text{-x} = 4 + 1 = 5$
 $rw\text{-} = 4 + 2 = 6$ $r\text{--} = 4$

Example: 640 = Owner(6=rw-) Group(4=r--) Others(0=---)

To directly observe the risk of over-permissioning, `chmod 777` was applied to `testfile.txt`. The first attempt, run without `sudo`, failed.

To directly observe the risk of over-permissioning, `chmod 777` was applied to `testfile.txt`. The first attempt, run without `sudo`, failed.

With `777` applied, every account on the system owner, group and others alike gained full read, write and execute access to the file. The permission was then restored to the least-privilege `640` configuration used earlier



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ stat -c '%A %a %u %g %n' ~/CyberSecurityLab/testfile.txt  
-rw-r----- 640 labadmin labusers /home/kali/CyberSecurityLab/testfile.txt  
  
(kali@kali)-[~]  
$ chmod 777 ~/CyberSecurityLab/testfile.txt  
chmod: changing permissions of '/home/kali/CyberSecurityLab/testfile.txt': Operation not permitted  
  
(kali@kali)-[~]  
$ sudo chmod 777 ~/CyberSecurityLab/testfile.txt  
[sudo] password for kali:  
  
(kali@kali)-[~]  
$ ls -l ~/CyberSecurityLab/testfile.txt  
-rwxrwxrwx 1 labadmin labusers 35 Sep  7 07:24 /home/kali/CyberSecurityLab/testfile.txt  
  
(kali@kali)-[~]  
$ sudo chmod 640 ~/CyberSecurityLab/testfile.txt  
  
(kali@kali)-[~]  
$ chmod 640 ~/CyberSecurityLab/testfile.txt  
chmod: changing permissions of '/home/kali/CyberSecurityLab/testfile.txt': Operation not permitted  
  
(kali@kali)-[~]  
$ sudo chmod 640 ~/CyberSecurityLab/testfile.txt  
  
(kali@kali)-[~]  
$ stat -c '%A %a %u %g %n' ~/CyberSecurityLab/testfile.txt  
-rw-r----- 640 labadmin labusers /home/kali/CyberSecurityLab/testfile.txt  
  
(kali@kali)-[~]  
$
```

testfile.txt made world-writable (777), then restored

Security risk

A world-writable resource allows unrelated local users to modify it.

For sensitive resources, this can result in:

- unauthorized modification,
- data tampering,
- malicious content insertion,
- execution-related risks for executable files,
- violation of least privilege.

The permission was subsequently restored: `sudo chmod 640 ~/CyberSecurityLab/testfile.txt`

Final state: `640`

Q.why should chmod 777 generally be avoided for sensitive resources?

chmod 777 grants read, write and execute to every account on the system, including accounts that have no legitimate reason to touch the resource at all. For a data file, this means any local user can silently modify or overwrite its contents; for an executable, it additionally means any local user could replace the program itself with malicious code that will then run with whatever privilege the next invoker has. It directly violates least privilege, and in a multi-user or shared environment it removes any meaningful separation between accounts for that resource. The correct approach, demonstrated here, is to grant only the specific owner/group/others

permissions actually required - 640 in this case , rather than defaulting to the broadest possible setting to avoid troubleshooting a permission error.

8.Part F-Special Permissions: SUID, SGID and Sticky Bit

Beyond the ordinary read/write/execute bits, Linux provides three special permission mechanisms that change how a file or directory behaves for other users. Each was examined in turn.

SUID (Set User ID) — numeric value 4000

When set on an executable, SUID causes the program to run with the effective privileges of the file's owner rather than the privileges of the user who launched it.

The existence of an SUID executable is not automatically a vulnerability every program in this list has a legitimate reason to elevate briefly (changing a password, mounting a filesystem, switching users). The risk is that if a privileged SUID program contains a vulnerability or is poorly written, that flaw can be exploited to gain the elevated privilege the program was trusted with, which is why the set of SUID binaries on a system is a standard item to review during a security audit.

SGID (Set Group ID) — numeric value 2000

On an executable, SGID causes the program to run with the effective group of the file rather than the invoking user's group. On a directory, SGID instead causes new files and subdirectories created inside it to automatically inherit the directory's group, which is useful for collaborative directories shared by a team.

As with SUID, unnecessary SGID permissions widen the privileged attack surface, since a vulnerability in any one of these binaries would expose whatever group privilege it runs with.

Security risk

Unnecessary SGID permissions can increase the privileged attack surface.

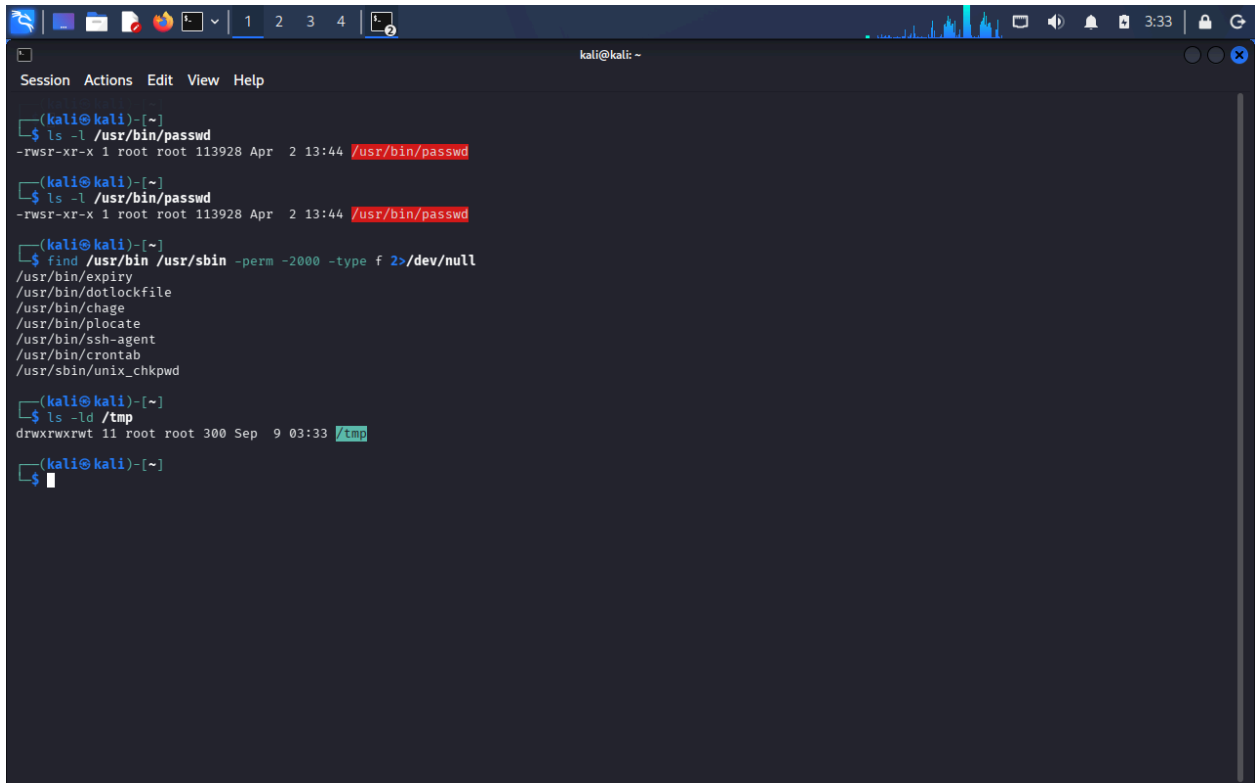
Sticky Bit — numeric value 1000

The sticky bit is applied to directories rather than files, and its classic use is on world-writable shared directories such as /tmp.

The t in the final position of drwxrwxrwt is the sticky bit. Any user can create a file inside a sticky-bit directory, but with the sticky bit set, only the file's own owner (or root) can delete or rename it, even though the directory itself is world-writable. Without the sticky bit, a world-writable /tmp would let any local user delete or rename files that belong to someone else the sticky bit is what makes a shared, writable-by-everyone directory safe to use at all.

Security benefit :Allows controlled shared use of directories such as `/tmp`.

Potential risk: A world-writable directory without appropriate sticky-bit protection can permit destructive file deletion/renaming by other users.

A terminal window titled 'kali@kali: ~' showing several commands and their outputs. The first command is 'ls -l /usr/bin/passwd', which outputs '-rwsr-xr-x 1 root root 113928 Apr 2 13:44 /usr/bin/passwd'. The second command is 'ls -l /usr/bin/passwd', which outputs the same line. The third command is 'find /usr/bin /usr/sbin -perm -2000 -type f 2>/dev/null', which outputs a list of files: /usr/bin/expiry, /usr/bin/dotlockfile, /usr/bin/chage, /usr/bin/plocate, /usr/bin/ssh-agent, /usr/bin/crontab, and /usr/sbin/unix_chkpwd. The fourth command is 'ls -ld /tmp', which outputs 'drwxrwxrwt 11 root root 300 Sep 9 03:33 /tmp'. The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The top bar shows 'kali@kali: ~' and a status bar with '3:33' and a lock icon.

```
kali@kali: ~  
$ ls -l /usr/bin/passwd  
-rwsr-xr-x 1 root root 113928 Apr  2 13:44 /usr/bin/passwd  
  
$ ls -l /usr/bin/passwd  
-rwsr-xr-x 1 root root 113928 Apr  2 13:44 /usr/bin/passwd  
  
$ find /usr/bin /usr/sbin -perm -2000 -type f 2>/dev/null  
/usr/bin/expiry  
/usr/bin/dotlockfile  
/usr/bin/chage  
/usr/bin/plocate  
/usr/bin/ssh-agent  
/usr/bin/crontab  
/usr/sbin/unix_chkpwd  
  
$ ls -ld /tmp  
drwxrwxrwt 11 root root 300 Sep  9 03:33 /tmp  
  
$
```

9. PART G - LINUX CAPABILITIES

Linux capabilities split the traditionally all-or-nothing privileges of root into a large number of smaller, independently assignable units, so that a program can be given exactly the one privileged operation it needs for example the ability to open a raw network socket , without being handed unrestricted root authority.

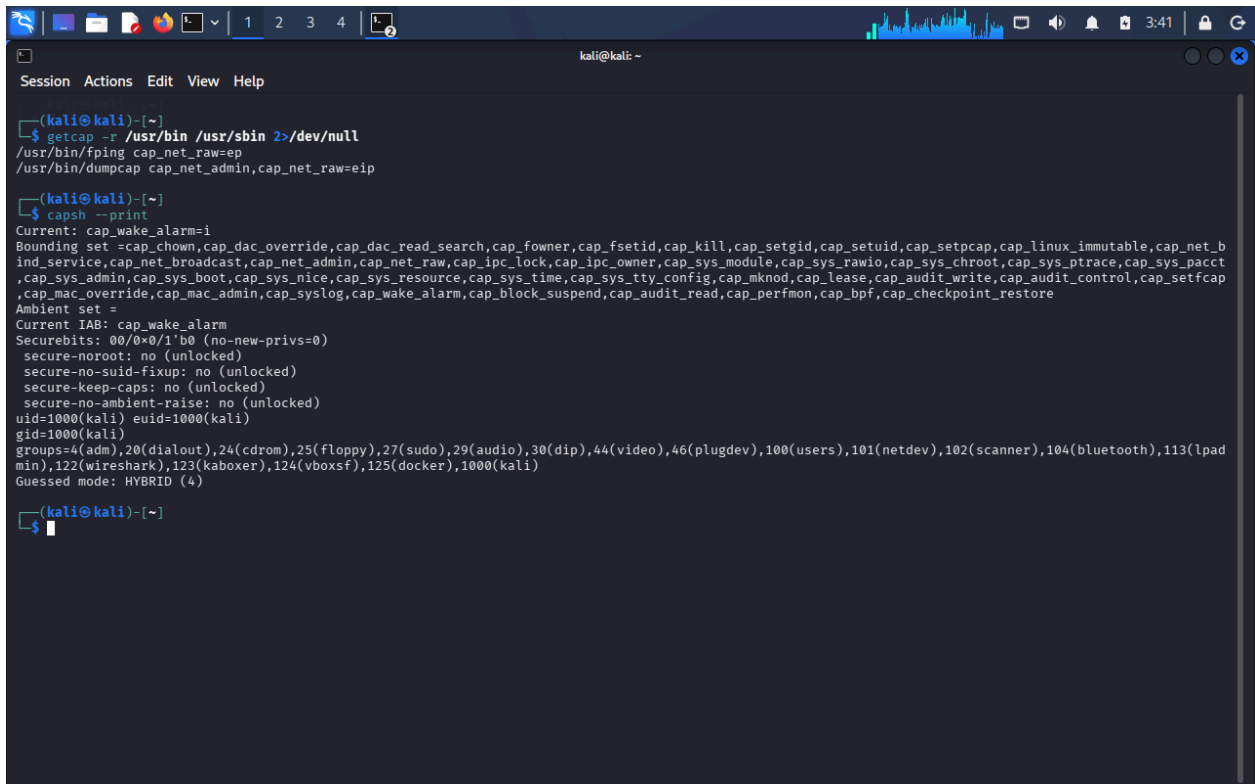
fping and dumpcap are both network-diagnostic tools that need to construct or capture raw packets a privileged operation that would traditionally have required running the tool as root, or making it SUID root. Assigning `cap_net_raw` directly instead means the tool can do exactly that one privileged thing and nothing else, which is a meaningfully smaller grant of trust than full root.

Interpreting the flags

- CAP_NET_RAW - permits the creation of raw and packet sockets, used for tools that need to see or forge network packets directly.
- CAP_NET_ADMIN - permits broader network-administration operations such as changing interface configuration and routing.
- e / p / i - the Effective, Permitted and Inheritable capability sets. cap_net_raw=ep means the capability is present in both the effective set (active right now) and the permitted set (allowed to be used); eip additionally marks it as inheritable across an exec() call.

capsh --print

A capability being listed in the bounding set does not mean the current, ordinary shell is actually exercising it the bounding set is only the ceiling of what a process running as this user could ever be granted; the current (active) set for an ordinary login shell is empty, since capabilities beyond what a normal unprivileged process needs are not active by default



```

(kali@kali)-[~]
$ getcap -r /usr/bin /usr/sbin 2>/dev/null
/usr/bin/fping cap_net_raw=ep
/usr/bin/dumpcap cap_net_admin,cap_net_raw=eip

(kali@kali)-[~]
$ capsh --print
Current: cap_wake_alarm=
Bounding set =cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_linux_immutable,cap_net_b
ind_service,cap_net_broadcast,cap_net_admin,cap_net_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_ptrace,cap_sys_pacct
,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,cap_audit_control,cap_setfcap
,cap_mac_override,cap_mac_admin,cap_syslog,cap_wake_alarm,cap_block_suspend,cap_audit_read,cap_perfmon,cap_bpf,cap_checkpoint_restore
Ambient set =
Current IAB: cap_wake_alarm
Securebits: 00/0x0/1'b0 (no-new-privs=0)
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
secure-no-ambient-raise: no (unlocked)
uid=1000(kali) euid=1000(kali)
gid=1000(kali)
groups=4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(scanner),104(bluetooth),113(lpda
min),122(wireshark),123(kaboxer),124(vboxsf),125(docker),1000(kali)
Guessed mode: HYBRID (4)

(kali@kali)-[~]
$

```

Why Capabilities Are Preferable to Unrestricted Root

Traditional root privilege is a single, enormous bundle: a process either has it all, or none of it. Capabilities decompose that bundle into dozens of individually assignable pieces, so an application can be given only the specific privileged operation its function actually requires. This directly supports the principle of least privilege: if a capability-limited application is later compromised, the attacker inherits only that narrow capability rather than full root, whereas an equivalent SUID-root or root-run program would have handed over everything.

Q: How does the Linux capability model support the principle of least privilege?

A: By replacing an all-or-nothing root grant with a fine-grained set of independently assignable privileges, capabilities let each application be given exactly the specific privileged operation it needs CAP_NET_RAW for a packet-capture tool, for instance — and nothing beyond that. This bounds the impact of a vulnerability in that application to whatever narrow capability it was actually granted, rather than to the full authority of root, which is the essence of least privilege applied at the level of individual programs rather than individual users. It is worth noting, however, that unnecessary capabilities carry a real risk of their own: if a vulnerable application has been granted a powerful capability it does not truly need, exploiting that application still hands the attacker that privileged operation, so capabilities reduce but do not eliminate the need for careful configuration.

10.Part H - Shells: Bash and Zsh

The shell is the command-line interface between the user and the operating system kernel. Kali configures Zsh as the default interactive shell, which was confirmed directly.

The current shell was determined using:

```
echo $SHELL
```

The observed shell was: zsh

The active shell process was also verified using:

```
ps -p $$ -o comm=
```

which returned: zsh

Feature	Bash (Bourne Again Shell)	Zsh (Z Shell)
Command execution	Standard POSIX-compatible execution	POSIX-compatible, with additional syntax extensions
Variables	Standard shell variables and arrays	Standard variables plus richer array handling
History	Command history with basic search	History with powerful incremental search and sharing across sessions
Tab completion	Good, extensible via bash-completion	Highly extensible; context-aware completion out of the box
Scripting	The de-facto standard for shell scripts	Supports scripting, though Bash remains more portable for scripts
Customization	Extensive via .bashrc	Very extensive via .zshrc and frameworks such as Oh My Zsh

Security posture	Depends on configuration and the commands run	Depends on configuration and the commands run
------------------	---	---

Security consideration

A shell itself does not bypass Linux access controls. A command executed through Zsh is still subject to:

- UID
- GID
- file permissions
- capabilities
- sudo policy
- other security mechanisms

Therefore, shell choice primarily affects the user interface and scripting environment rather than replacing the underlying Linux authorization model.

11. Part I : Security-Relevant Linux Commands

The commands specified by the assignment were grouped into the same eight categories used in the assignment brief. For each command, the table records its purpose, an example invocation (with the more important options), and its relevance to a security assessment.

System Information

Command	Purpose	Example / Key Options	Security Relevance
uname	Reports kernel and system information.	uname -a (full); uname -r (kernel release only)	Kernel version can be checked against known vulnerabilities; also useful for system fingerprinting.
hostname	Displays the machine's network hostname.	hostname	Basic system identification, used to correlate logs and observations across machines.
uptime	Shows how long the system has been running, load average and logged-in user count.	uptime	Unexpectedly long or short uptime, or an unfamiliar load pattern, can be a starting point for further investigation.

lsb_release	Reports distribution name and version.	lsb_release -a	Confirms exactly which distribution and release is running, relevant to patch and advisory tracking.
-------------	--	----------------	--

User and Identity

Command	Purpose	Example / Key Options	Security Relevance
whoami	Prints the current effective username.	whoami	Confirms which identity a shell or script is actually operating as.
id	Prints UID, GID and all group memberships.	id; id <username>	Central command for privilege analysis — reveals every group, and therefore every extra access right, an account holds.
who	Lists currently logged-in users and their session origin.	who	Session monitoring; an unexpected login source is a red flag.
w	Lists logged-in users together with their current activity.	w	Extends who with what each session is doing, useful for spotting suspicious activity in real time.
groups	Lists the groups a user belongs to.	groups <username>	A quick privilege check without the full id output.

Files and Permissions

Command	Purpose	Example / Key Options	Security Relevance
ls	Lists directory contents, including permissions and ownership.	ls -l; ls -ld <dir>	The starting point for any access-control audit of a file or directory.
stat	Displays detailed file metadata.	stat -c '%A %a %U %G %n' <file>	Confirms exact permission bits, owner and group in one line, useful for scripted audits.
chmod	Changes permission bits.	chmod 640 <file>; chmod u+x <file>	The primary access-control configuration command — and, misused, the primary source of over-permissioning.
chown	Changes file ownership.	sudo chown <user> <file>	Ownership changes affect who is evaluated as the file's "owner" for permission purposes; restricted to root for that reason.

chgrp	Changes the group associated with a file.	sudo chgrp <group> <file>	Group-based access control depends entirely on correct group ownership.
getfacl	Displays POSIX Access Control Lists.	getfacl <file>	Reveals extended permissions that a plain ls -l listing cannot show.

Processes

Command	Purpose	Example / Key Options	Security Relevance
ps	Lists running processes.	ps aux	Identifies unexpected processes, and which account (often root) each one runs as.
top	Live view of processes and resource usage.	top	Surfaces abnormal CPU/memory consumption that may warrant investigation.
pgrep	Finds processes matching a name pattern.	pgrep -a <name>	Quickly confirms whether a specific process is running, without scrolling through ps output.
kill	Sends a signal to a process.	kill -9 <pid>	Legitimate for stopping a runaway process, but the same capability could stop a security tool if the invoking account has sufficient rights.

Services

Command	Purpose	Example / Key Options	Security Relevance
systemctl	Manages and inspects systemd units and services.	systemctl status; systemctl list-units --type=service	Central tool for attack-surface assessment — every running service is something that must be secured.
journalctl	Reads the systemd journal (system and kernel logs).	journalctl -k; journalctl -b	Primary log-analysis tool for security investigation and incident response on a systemd-based system.

Networking

Command	Purpose	Example / Key Options	Security Relevance
ip	Displays or configures network interfaces, addresses and routes.	ip addr; ip route	Reveals active interfaces, addresses and connectivity — useful for both administration and reconnaissance.

ss	Displays socket and listening-port information.	ss -tulnp	Fast way to spot an unexpected listening service or exposed port.
ping	Tests reachability using ICMP.	ping -c 4 <host>	Basic connectivity diagnostics; also usable, at scale, for network sweeps.
traceroute	Shows the hop-by-hop path to a destination.	traceroute <host>	Reveals network topology and intermediate routing devices.
nslookup	Performs a DNS lookup.	nslookup <host>	Confirms whether DNS resolution is behaving as expected, or has been tampered with.

Disk and Storage

Command	Purpose	Example / Key Options	Security Relevance
df	Reports filesystem space usage.	df -h	Unexpected disk usage can indicate log tampering, data staging, or resource exhaustion.
du	Reports directory space usage.	du -sh <dir>	Helps localise which directory is responsible for unusual disk consumption.
mount	Lists or performs filesystem mounts.	mount grep <fs>	Reveals what storage, including removable or network storage, is currently attached.
lsblk	Lists block devices and partitions.	lsblk	Storage/partition auditing, including detection of unexpected attached devices.

Search and Text Processing

Command	Purpose	Example / Key Options	Security Relevance
find	Searches the filesystem by name, permission, type and more.	find / -perm -4000 -type f	Used throughout this report to discover SUID/SGID files and world-writable resources.
grep	Searches text for a pattern.	grep -i "failed" /var/log/auth.log	Core tool for log analysis and searching configuration for insecure settings.

cat	Prints a file's contents.	cat <file>	Simple and effective — a reminder that read access alone is often enough to expose sensitive data.
less	Interactively views a file, page by page.	less <file>	Practical for inspecting large log files without loading the whole file into a terminal buffer.
head	Prints the first lines of a file.	head -n 20 <file>	Quick inspection of a file or log without opening it fully.
tail	Prints the last lines of a file, optionally following new output.	tail -f <logfile>	Standard way to monitor a log file in real time.

Selected Command Demonstrations

A subset of the commands above was demonstrated in full, with actual system output recorded, to illustrate their practical use during a security review of this VM.

1.Process analysis using ps aux

The presence of numerous processes running as root — the init process, kernel worker threads and several system services — is normal and expected: these are core operating-system functions that require root by design. ps is useful during a review for the opposite reason: to notice a process that should not be running as root, or that has an unfamiliar name or command line

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
└─$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.6 25984 13992 ?        Ss   02:39   0:04 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    02:39   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    02:39   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-netns]
root          13  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-mm_percpu_wq]
root          14  0.0  0.0      0     0 ?        S    02:39   0:03 [ksoftirqd/0]
root          15  0.0  0.0      0     0 ?        I    02:39   0:03 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    02:39   0:00 [rcu_exp_par_gp_kthread_worker/0]
root          17  0.0  0.0      0     0 ?        S    02:39   0:01 [rcu_exp_gp_kthread_worker]
root          18  0.0  0.0      0     0 ?        S    02:39   0:00 [migration/0]
root          19  0.0  0.0      0     0 ?        S    02:39   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    02:39   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    02:39   0:00 [cpuhp/1]
root          22  0.0  0.0      0     0 ?        S    02:39   0:00 [idle_inject/1]
root          23  0.0  0.0      0     0 ?        S    02:39   0:00 [migration/1]
root          24  0.2  0.0      0     0 ?        S    02:39   0:09 [ksoftirqd/1]
root          29  0.0  0.0      0     0 ?        S    02:39   0:00 [kdevtmpfs]
root          30  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-inet_frag_wq]
root          31  0.0  0.0      0     0 ?        I    02:39   0:00 [rcu_tasks_kthread]
root          32  0.0  0.0      0     0 ?        I    02:39   0:00 [rcu_tasks_rude_kthread]
root          33  0.0  0.0      0     0 ?        I    02:39   0:00 [rcu_tasks_trace_kthread]
root          34  0.0  0.0      0     0 ?        S    02:39   0:00 [kauditd]
root          35  0.0  0.0      0     0 ?        S    02:39   0:00 [khungtaskd]
root          36  0.0  0.0      0     0 ?        S    02:39   0:00 [oom_reaper]
root          38  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-writeback]
root          39  0.0  0.0      0     0 ?        S    02:39   0:02 [kcompactd0]
root          40  0.0  0.0      0     0 ?        SN   02:39   0:00 [ksmd]
root          41  0.0  0.0      0     0 ?        SN   02:39   0:00 [khugepaged]
root          42  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-kblockd]
root          43  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-blkcg_punt_bio]
root          44  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-kintegrityd]
root          45  0.0  0.0      0     0 ?        S    02:39   0:00 [irq/9-acpi]
root          47  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-tpm_dev_wq]
root          48  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-edac-poller]
root          49  0.0  0.0      0     0 ?        I<   02:39   0:00 [kworker/R-devfreq_wq]

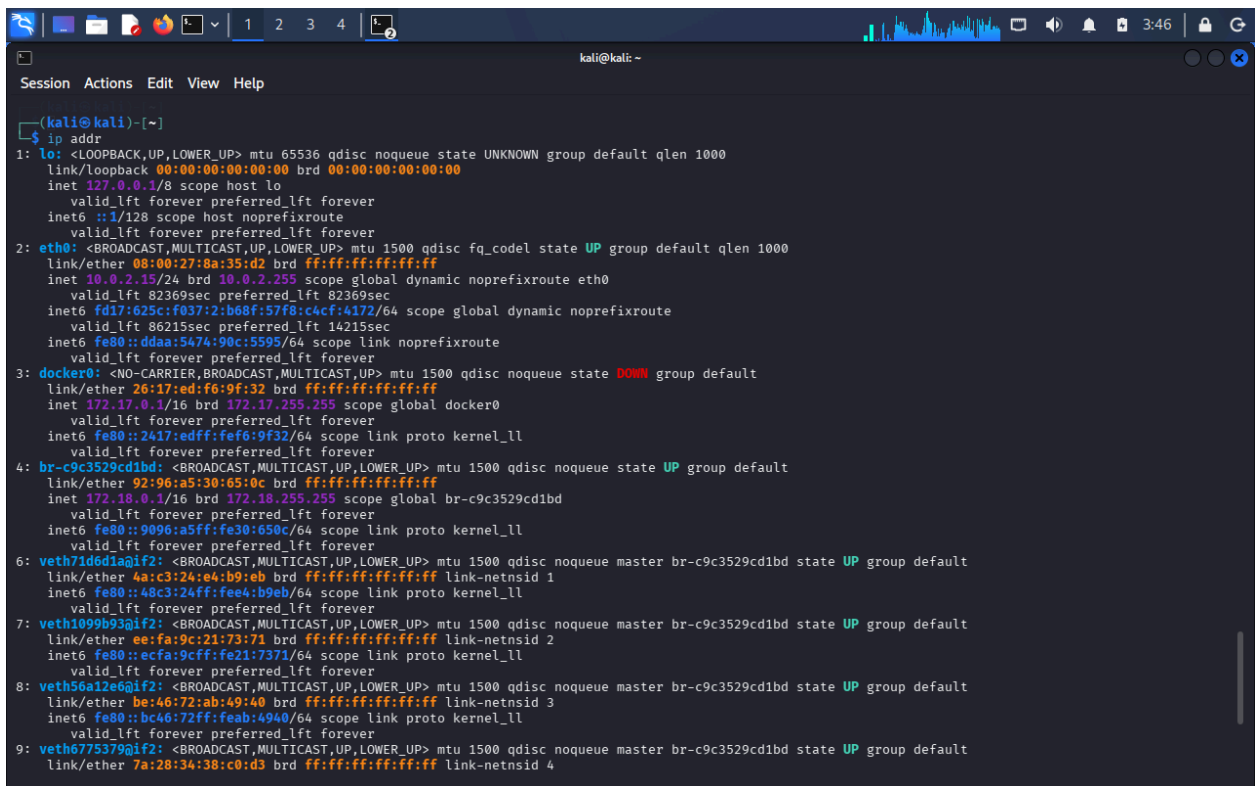
```

2.Resource analysis using top

233 total tasks were observed, with 229 sleeping and one zombie process. A zombie process is simply a terminated process whose parent has not yet collected its exit status; it is not, by itself, a security concern. dockerd's comparatively high CPU usage is explained by the Docker/containerd activity also visible in the systemd

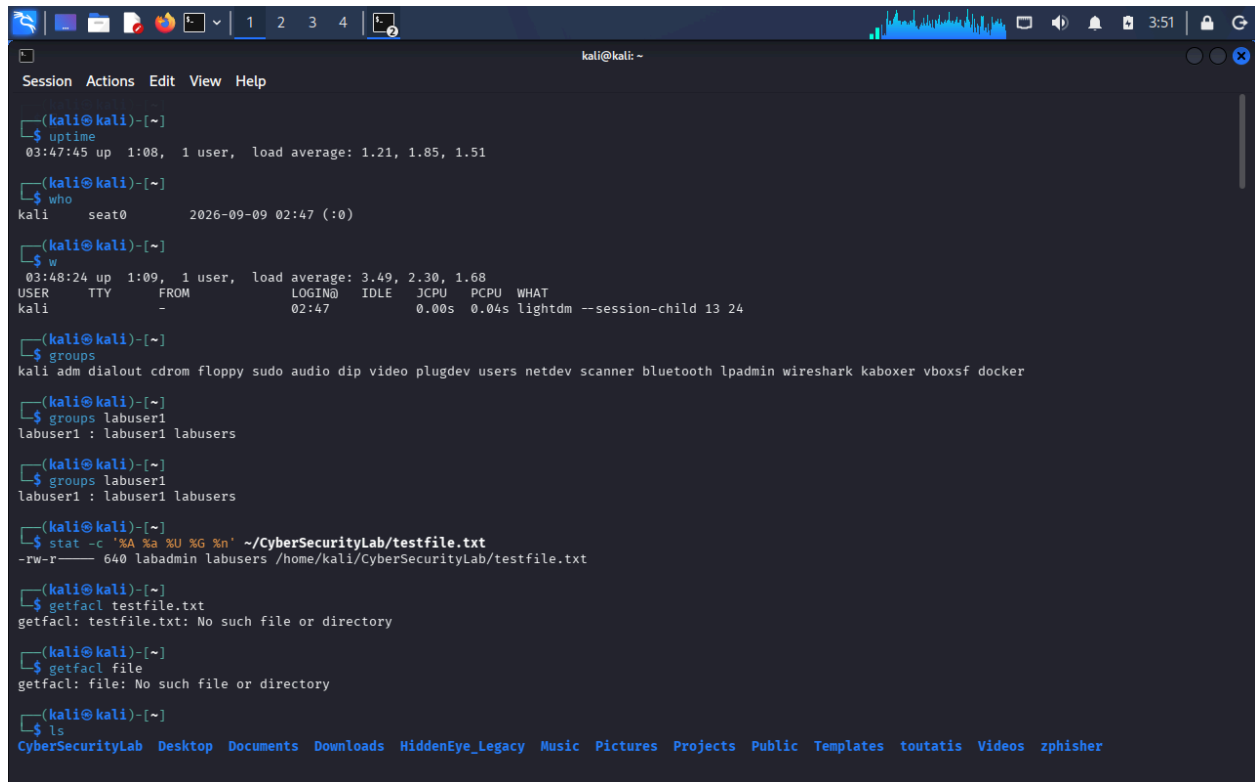
3.Network Analysis Using ip addr

The NAT-assigned address 10.0.2.15/24 on eth0 is the VirtualBox default NAT network range, confirming the VM is isolated behind VirtualBox's NAT rather than bridged directly onto the host network. The additional 172.17.0.1/16 and 172.18.0.1/16 addresses belong to Docker's own internal bridge networks, created automatically once the docker service started. This illustrates how network-enumeration commands, while entirely legitimate for administration, also reveal information active interfaces, address ranges, and evidence of container tooling that would be useful to an attacker performing reconnaissance.



```
(kali@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 82369sec preferred_lft 82369sec
    inet6 fd17:625c:f037:2:b68f:57f8:c4cf:4172/64 scope global dynamic noprefixroute
        valid_lft 86215sec preferred_lft 14215sec
    inet6 fe80::d8aa:5474:90c:5595/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 26:17:ed:f6:9f:32 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::2417:edff:fe6:9f32/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
4: br-c9c3529cd1bd: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 92:96:a5:30:65:0c brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-c9c3529cd1bd
        valid_lft forever preferred_lft forever
    inet6 fe80::9096:a5ff:fe30:650c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
6: veth71d6d1a91f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-c9c3529cd1bd state UP group default
    link/ether 4a:c3:24:e4:b9:eb brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::48c3:24ff:fe4:b9eb/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
7: veth1099b93d1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-c9c3529cd1bd state UP group default
    link/ether ee:fa:9c:21:73:71 brd ff:ff:ff:ff:ff:ff link-netnsid 2
    inet6 fe80::ecfa:9cff:fe21:7371/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
8: veth56a12e601f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-c9c3529cd1bd state UP group default
    link/ether be:46:72:ab:49:40 brd ff:ff:ff:ff:ff:ff link-netnsid 3
    inet6 fe80::bc46:72ff:feab:4940/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
9: veth6775379d1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-c9c3529cd1bd state UP group default
    link/ether 7a:28:34:38:c0:d3 brd ff:ff:ff:ff:ff:ff link-netnsid 4
```

4.Session analysis using uptime, who, w & getfacl



```
(kali@kali)-[~]
$ uptime
03:47:45 up 1:08, 1 user, load average: 1.21, 1.85, 1.51

(kali@kali)-[~]
$ who
kali    seat0      2026-09-09 02:47 (:0)

(kali@kali)-[~]
$ w
03:48:24 up 1:09, 1 user, load average: 3.49, 2.30, 1.68
USER    TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
kali    -        -             02:47    0.00s  0.04s  lightdm --session-child 13 24

(kali@kali)-[~]
$ groups
kali adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer vboxsf docker

(kali@kali)-[~]
$ groups labuser1
labuser1 : labuser1 labusers

(kali@kali)-[~]
$ groups labuser1
labuser1 : labuser1 labusers

(kali@kali)-[~]
$ stat -c '%A %a %U %G %n' ~/CyberSecurityLab/testfile.txt
-rw-r--r-- 640 labadmin labusers /home/kali/CyberSecurityLab/testfile.txt

(kali@kali)-[~]
$ getfacl testfile.txt
getfacl: testfile.txt: No such file or directory

(kali@kali)-[~]
$ getfacl file
getfacl: file: No such file or directory

(kali@kali)-[~]
$ ls
CyberSecurityLab Desktop Documents Downloads HiddenEye_Legacy Music Pictures Projects Public Templates toutatis Videos zphisher
```

Getfacl - the ACL information was displayed.

The output showed the owner and group information and the effective basic permissions.

Security significance-

getfacl is important because traditional **ls -l** output may not reveal every additional ACL entry when extended ACLs are configured.

5.journalctl

The kernel version in this particular journal excerpt reads 6.18.12+kali-amd64, one point release older than the 6.19.14+kali-amd64 reported by the live `uname -a` in Section 3.2. This is not an inconsistency: `journalctl` retains logs from previous boots, and the excerpt above is timestamped June 04, well before the current session on September 09, so it simply reflects the kernel that was running before a subsequent update. This is itself a useful demonstration of why timestamps and boot context must always be checked first during log or forensic analysis, since two genuinely accurate readings from the same machine can otherwise appear to contradict each other.

```
kali@kali: ~  
$ journalctl  
Jun 04 07:44:57 kali kernel: Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.0, GNU ld (GNU Binutils fo  
Jun 04 07:44:57 kali kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-6.18.12+kali-amd64 root=UUID=894e89ec-b178-42b7-a450-6e969a2aa8fe ro quiet splash  
Jun 04 07:44:57 kali kernel: [Firmware Bug]: TSC doesn't count with P0 frequency!  
Jun 04 07:44:57 kali kernel: BIOS-provided physical RAM map:  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x0000000000100000-0x00000000007fffff] usable  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x0000000007fff0000-0x0000000007fffff] ACPI data  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved  
Jun 04 07:44:57 kali kernel: BIOS-e820: [mem 0x00000000fffc0000-0x00000000fffcfffff] reserved  
Jun 04 07:44:57 kali kernel: NX (Execute Disable) protection: active  
Jun 04 07:44:57 kali kernel: APIC: Static calls initialized  
Jun 04 07:44:57 kali kernel: SMBIOS 2.5 present.  
Jun 04 07:44:57 kali kernel: DMI: innotek GmbH VirtualBox/VirtualBox, BIOS VirtualBox 12/01/2006  
Jun 04 07:44:57 kali kernel: DMI: Memory slots populated: 0/0  
Jun 04 07:44:57 kali kernel: Hypervisor detected: KVM  
Jun 04 07:44:57 kali kernel: last_pfn = 0x80000 max_arch_pfn = 0x400000000  
Jun 04 07:44:57 kali kernel: kvm-clock: Using msrs 4b564d01 and 4b564d00  
Jun 04 07:44:57 kali kernel: kvm-clock: using sched offset of 22278588678 cycles  
Jun 04 07:44:57 kali kernel: clocksource: kvm-clock: mask: 0xfffffffffffff max_cycles: 0x1cd42e4dffb, max_idle_ns: 881590591483 ns  
Jun 04 07:44:57 kali kernel: tsc: Detected 2894.594 MHz processor  
Jun 04 07:44:57 kali kernel: e820: update [mem 0x00000000-0x00000ffff] usable ==> reserved  
Jun 04 07:44:57 kali kernel: e820: remove [mem 0x000a0000-0x000ffff] usable  
Jun 04 07:44:57 kali kernel: last_pfn = 0x80000 max_arch_pfn = 0x400000000  
Jun 04 07:44:57 kali kernel: MTRR map: 3 entries (3 fixed + 0 variable; max 19), built from 8 variable MTRRs  
Jun 04 07:44:57 kali kernel: x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT  
Jun 04 07:44:57 kali kernel: CPU MTRRs all blank - virtualized system.  
Jun 04 07:44:57 kali kernel: found SMP MP-table at [mem 0x0009fbf0-0x0009fbff]  
Jun 04 07:44:57 kali kernel: RAMDISK: [mem 0x1f375000-0x2b9b1fff]  
Jun 04 07:44:57 kali kernel: ACPI: Early table checksum verification disabled  
Jun 04 07:44:57 kali kernel: ACPI: RSDP 0x000000000000E000 000024 (v02 VBOX )  
Jun 04 07:44:57 kali kernel: ACPI: XSDT 0x000000007FFF0030 00003C (v01 VBOX VBOXXSDT 00000001 ASL 00000061)  
Jun 04 07:44:57 kali kernel: ACPI: FACP 0x000000007FFF00F0 0000F4 (v04 VBOX VBOXFACP 00000001 ASL 00000061)  
Jun 04 07:44:57 kali kernel: ACPI: DSDT 0x000000007FFF02F0 002353 (v02 VBOX VBOXBIOS 00000002 VBOX 0002A6A2)  
Jun 04 07:44:57 kali kernel: ACPI: FACS 0x000000007FFF0200 000040  
Jun 04 07:44:57 kali kernel: ACPI: FACS 0x000000007FFF0200 000040  
Jun 04 07:44:57 kali kernel: ACPI: APIC 0x000000007FFF0240 00005C (v02 VBOX VBOXAPIC 00000001 ASL 00000061)  
Jun 04 07:44:57 kali kernel: ACPI: SSDT 0x000000007FFF02A0 000045 (v01 VBOX VBOXCPU0 00000002 VBOX 0002A6A2)
```

12. PART J - AUTHENTICATION, AUTHORIZATION AND PRIVILEGE

Authentication answers who are you, and is established at login through a username and password (or another credential) checked against the account database. Authorization is a separate, later question now that the system knows who you are, what are you actually permitted to do and on Linux this is decided continuously, every time a UID/GID is checked against a file's permission bits, a capability set, or the sudoers policy. This is the same distinction studied for Windows in Lab Assignment 1, where authentication happened once at logon against the SAM database and authorization was then re-evaluated against each resource's ACL.

root

The root account holds UID 0 and is exempt from almost every normal permission check, giving it essentially unrestricted authority over the system's files, processes, users and configuration. Because of this breadth of power, routine work should never be performed logged in directly as root.

sudo

sudo allows a user who is authorised to do so (in this case, any member of the sudo group) to run an individual command with elevated privileges, without ever opening a persistent root shell.

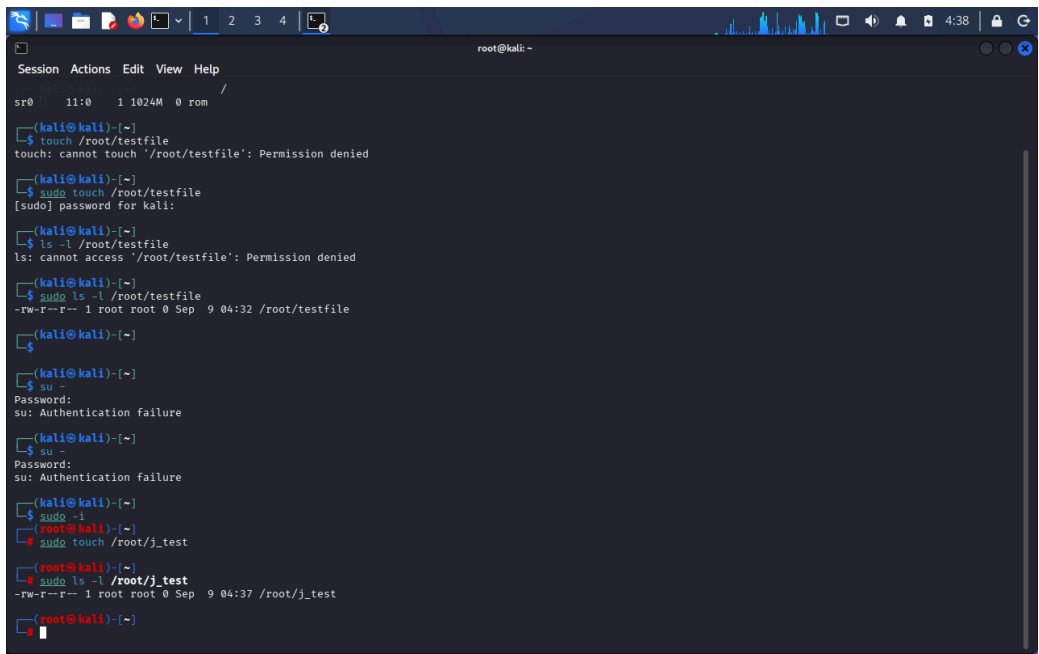
The labadmin account was placed in the sudo group for exactly this purpose, and this is reflected directly in /etc/group:

su versus sudo -i

The distinction between su and sudo was demonstrated directly. su - was attempted first, requesting a full root login shell:

This failed because su - authenticates against the target account's own password root's which was never set on this VM (a deliberate and common security posture on Debian-derived systems, where the root account is left locked and administration is expected to go through sudo instead). sudo -i was then used in its place:

sudo -i succeeded because it authenticates with the invoking user's own password (kali's), and then checks whether kali is authorised by the sudo policy to become root which it is, by virtue of sudo group membership rather than requiring knowledge of root's password at all.



Execution Method	Privilege Level	Security Characteristic
Ordinary user	Limited to the user's own UID/GID and group rights	Least privilege by default
sudo <command>	Elevated, but only for the single command invoked	Controlled, auditable elevation
sudo -i / root shell	Full root (UID 0), for the duration of the session	Broad and continuous privilege — used sparingly

Q: Why is root considered highly privileged?

root carries UID 0, which the kernel treats as exempt from almost every ordinary permission check. A process running as root can read or write any file regardless of its permission bits, manage any other process, load kernel modules, and reconfigure the system at will. A compromise of root is therefore, in practical terms, a compromise of the entire machine.

Q: Why is sudo preferable to routinely logging in as root?

sudo confines elevation to the single command that actually needs it, leaves a log entry recording which user ran which privileged command, and means the account is only ever briefly and deliberately elevated rather than continuously exposed at full privilege. Working as root by default removes every one of those safeguards at once.

Q: How does this compare with Windows administrator privileges and UAC?

The underlying goal is identical: avoid a session running with full privilege by default, but the mechanisms differ. UAC issues an administrator account two tokens and silently uses the filtered, standard-user token for everyday processes, only attaching the full administrator token to a specific process after an explicit prompt. sudo instead re-authenticates the user (or checks a cached credential) at the moment a privileged command is actually run. Windows makes elevation visible through a graphical consent prompt; Linux makes it visible through the terminal invocation of sudo itself and, typically, a syslog entry. Both achieve the same net effect: privilege is attached to a specific action, not to the whole session.

Q: How does the principle of least privilege apply in both operating systems?

A: In both cases, the account used for day-to-day work is deliberately kept at standard privilege, and elevated authority is requested only for the specific operation that requires it, for the shortest practical time. Lab Assignment 1 demonstrated this through LabAdmin's UAC-gated elevation and the separation between LabAdmin and the standard Student1/Student2 accounts; this lab demonstrates the same principle through labadmin's sudo membership being kept separate from the unprivileged labuser1/labuser2 accounts, and through sudo's per-command elevation model.

13. Security Weaknesses Identified and Recommended Mitigations

The assignment requires at least five Linux security weaknesses, documented as Weakness → Cause → Security Impact → Detection Method → Mitigation. The five below were each observed directly during the experiments above.

Weakness	Cause	Security Impact	Detection Method	Recommended Mitigation
----------	-------	-----------------	------------------	------------------------

Excessive file permissions (chmod 777)	A permission is set more broadly than the task actually requires, often to avoid troubleshooting an access-denied error.	Any local account not only the intended user or group can read, modify or delete the resource.	find / -perm -0002 -type f or a plain ls -l on the resource	Grant the minimum permission the task needs; testfile.txt was restored to 640 in Section 8.1.
World-writable resources	A file or directory is left with the o+w (or full 777) bit set after testing or troubleshooting.	Any unrelated local user can modify or overwrite the resource's contents.	find / -perm -0002 -type f 2>/dev/null	Remove unnecessary write access with chmod o-w, and rely on group ownership instead of world-writable bits.
Excessive default group membership	The default kali account accumulates convenience-driven group memberships (sudo, docker, wireshark, vboxsf) over time.	A single compromised account inherits every privilege those groups carry administrative command execution, control of the Docker daemon, and raw packet capture among them.	id kali / groups kali, compared against a documented list of what the account actually needs.	Periodically audit group membership and remove memberships no longer required for the account's actual role.
Unnecessary or unaudited SUID/SGID binaries	Special permissions are applied to programs that need to perform a specific privileged operation on behalf of an ordinary user.	A vulnerability in a privileged SUID/SGID program has significantly greater impact than the same flaw in an ordinary program, since it executes with the file owner's (often root's) privilege.	find /usr/bin /usr/sbin -perm -4000 find /usr/bin /usr/sbin -perm -2000	Regularly review the SUID/SGID inventory, remove the bit from anything that does not genuinely need it, and keep the remaining privileged binaries patched.
Excessive administrative (sudo) privileges	A user is added to the sudo group for convenience rather than because their role specifically requires it.	If that account is compromised, the attacker can trivially elevate to root via sudo.	getent group sudo	Restrict sudo membership strictly to accounts that genuinely require administrative access, as was done here by keeping labuser1 and labuser2 out of sudo entirely.

Two further weaknesses are worth noting more briefly from the same experiments: the unnecessary Docker, PostgreSQL and other network-capable services observed running, which enlarge the attack surface if they are not actually required on this machine; and the excessive

capabilities identified on `fping` and `dumppcap`, which, while individually reasonable for those specific tools, would still need auditing on a hardened system to confirm they remain necessary.

14. Windows–Linux Comparative Analysis

Privileged account	Administrator	root
User identification	User account / SID	UID
Groups	Windows Groups (e.g. CyberLabUsers)	Linux Groups / GID (e.g. labusers)
File permissions	NTFS ACL	Owner / Group / Others, extendable with POSIX ACLs
Privilege elevation	UAC (token filtering + consent prompt)	<code>sudo</code> / <code>su</code> (per-command re-authentication)
Shell	CMD / PowerShell	Bash / Zsh
Fine-grained privilege	Windows security mechanisms / specific user rights	Linux Capabilities
Service management	Windows Services / Service Control Manager	<code>systemd</code>
Security logs	Event Logs	<code>journald</code> (<code>journalctl</code>)

Observations

1. Identity model In Lab Assignment 1, LabAdmin was identified internally by a Windows SID even though the report referred to it by name; here, kali and labadmin are identified internally by the numeric UIDs 1000 and 1002 respectively. Both systems separate the human-readable account name from the identifier actually used for access-control decisions.
2. Group-based access CyberLabUsers was created in Windows specifically to grant Student1 and Student2 shared access to StudentData without configuring each account individually; labusers plays exactly the same role here for labuser1 and labuser2. The mechanism (collective rights via group membership) is identical across both operating systems, even though the underlying data structure an ACL entry versus a permission-bit triple differs.
3. Permission enforcement across a path The StudentData/AdminData experiment in Lab Assignment 1 showed inherited NTFS permissions blocking access even where a subfolder's own permissions looked sufficient. The `namei` experiment in Section 6.4 of this lab showed the equivalent Linux behaviour from a different mechanism: labuser1 was blocked not by the file's own permissions but by a restrictive parent directory (`/home/kali`) earlier in the path.

Both experiments demonstrate the same underlying lesson permission on the target object alone is not sufficient; the whole access path must be evaluated.

4. Privilege elevation UAC's split-token design meant LabAdmin ran with a standard-user token by default and only gained an administrator token after an explicit consent prompt. sudo achieves a functionally similar outcome by default-denying elevated execution and requiring an explicit, authenticated sudo invocation for each privileged command. Both mechanisms exist specifically to prevent an administrative account from operating at full privilege by default.

5. Excessive privilege as the recurring weakness In Lab Assignment 1, the leading weakness identified was excessive Administrators-group membership; in this lab, the leading weaknesses were excessive file permissions (chmod 777) and excessive sudo/group membership. In both operating systems, the security mechanisms themselves (ACLs and UAC on Windows; permission bits, groups and sudo on Linux) proved effective when configured narrowly, and the actual weaknesses arose entirely from configuration choices layered on top of those mechanisms, not from any deficiency in the mechanisms themselves.

6. Auditability of commands whoami /groups and Get-Acl on Windows, and id and getfacl on Linux, serve the same auditing purpose: making the relationship between an identity and its granted access directly observable from the command line, which is what allowed every permission experiment in both labs to actually be verified rather than assumed.

15. Conclusion

This lab provided a hands-on study of Linux operating-system security, progressing from the boot process through user and group identity, file permissions, special permissions, capabilities, shells, security-relevant commands, and privilege elevation. Taken together with Lab Assignment 1, the experiments demonstrated that Windows and Linux implement access control through different concrete mechanisms while pursuing the identical underlying goal of restricting privilege to what is actually necessary.

The boot-sequence investigation traced the system from firmware through GRUB, the kernel, initramfs and systemd to the graphical login prompt, confirming systemd as PID 1 and graphical.target as the default boot target, and established why boot-chain integrity underpins every access-control mechanism that runs afterwards.

The user and group experiments showed that Linux relies on UID, GID and group membership as the basis for every access decision, in the same way Windows relies on account SIDs and Windows Groups. Creating labadmin, labuser1, labuser2 and labusers, and then tracing exactly why labuser1 was denied access to testfile.txt despite apparently sufficient group permissions, demonstrated a point that is easy to miss from theory alone: permission on the target file is necessary but not sufficient, since every directory between the filesystem root and that file must also grant traversal.

The chmod 777 experiment made the danger of excessive permissions directly observable, while the final restoration to 640 demonstrated the least-privilege alternative. SUID, SGID and the sticky bit were shown to be legitimate, necessary mechanisms passwd genuinely needs

SUID, and /tmp genuinely needs its sticky bit — whose risk lies not in their existence but in whether each specific grant is still required and correctly implemented. Linux capabilities carried the same idea further, breaking root's authority into pieces small enough that `fping` and `dumpcap` could each be granted only the one networking privilege they use.

The investigation of `sudo`, `su` and `root` made the practical difference between the two elevation paths concrete: `su` - failed because no root password had ever been set, while `sudo -i` succeeded on the strength of kali's own credentials and its `sudo` group membership a deliberate design choice that keeps administrative access tied to individually accountable users rather than to a shared root password.

Comparing this lab directly against Lab Assignment 1 showed that Windows and Linux are, in practice, closer to each other than their different terminology suggests: SID and UID, NTFS ACL and owner/group/others permissions, UAC and `sudo`, and Event Logs and `journald` each address the same problem from a different angle. In every weakness identified across both labs excessive Administrators-group membership on Windows, and excessive file permissions, group membership and `sudo` access on Linux the root cause was never a missing security feature; it was an available feature configured more broadly than the task at hand required.

Overall, the lab reinforced that least privilege is a configuration discipline that has to be actively maintained, not a property that a system acquires simply by having permissions, capabilities, `sudo`, SUID or ACLs available. The tools examined across both assignments `id`, `getfacl`, `find`, `systemctl`, `journalctl` on Linux; `whoami` /`groups`, `icacls`, `Get-Acl` on Windows exist precisely to make that configuration observable, so that the gap between "a mechanism is available" and "a mechanism is securely configured" can actually be checked rather than assumed.

16.Final observation based on assignment 1 & 2

Q: *Based on the experiments in Lab Assignments 1 and 2, which operating-system security mechanisms provide the strongest practical support for the principle of least privilege, and what configuration mistakes can weaken these mechanisms?*

A: Across both assignments, the mechanisms that gave the strongest practical support for least privilege were consistently the ones that let a privilege be scoped as narrowly as possible, rather than the ones that simply concentrated more authority in fewer hands. On Windows, that meant NTFS permissions and inheritance restricting AdminData to LabAdmin while leaving StudentData and SharedData open to the standard accounts through CyberLabUsers, and UAC's split-token model keeping even an administrator account running at standard privilege until a specific action was explicitly elevated. On Linux, the same role was filled by the owner/group/others permission model demonstrated most clearly by the `namei` experiment, where a single restrictive directory in the path was enough to override an apparently permissive file by `sudo`'s per-command elevation, and, at the finest grain observed in either lab, by

capabilities, which let `fping` and `dumpcap` be granted the one networking privilege they need instead of full root.

The configuration mistakes that weakened these mechanisms were remarkably consistent between the two operating systems even though the underlying tools were different. On Windows, the identified weaknesses were excessive Administrators-group membership, unnecessary or stale accounts, broken permission inheritance, and careless group assignment. On Linux, they were `chmod 777` and other over-broad file permissions, unnecessary accumulation of group memberships (`sudo`, `docker`, `wireshark` and `vboxsf` all sitting on a single default account), unaudited SUID/SGID binaries, and `sudo` access granted more widely than a role actually required. In every one of these cases the mechanism itself the ACL, the permission bit, the capability, the sudoers entry was fully capable of enforcing least privilege; the weakness was introduced afterwards, by a configuration choice made for convenience, made once during troubleshooting and never reverted, or simply never reviewed again.

The overall conclusion supported by both sets of experiments is therefore that least privilege is not a feature a system either has or lacks, but a property of how that system's existing features are configured and subsequently maintained. Windows and Linux each provide more than enough mechanism to enforce it ACLs, UAC, groups and auditing on one side, and permissions, `sudo`, capabilities and groups on the other and in both labs, every observed weakness traced back not to a missing control but to an existing control that had been configured more broadly than the task in front of it actually required.

17. References

1. Linux manual pages (man-db), consulted for `chmod`, `chown`, `chgrp`, `useradd`, `usermod`, `groupadd`, `systemctl`, `journalctl`, `ps`, `top`, `ip`, `ss`, `getfacl` and `capabilities(7)`.
2. Kali Linux Official Documentation, kali.org/docs — distribution-specific configuration and tooling reference.
3. GRUB Manual, GNU Project (gnu.org/software/grub) — bootloader configuration reference.
4. systemd System and Service Manager documentation, freedesktop.org — service and target reference used in Section 4.
5. Experimental observations and command output captured directly from the Kali Linux virtual machine described in Section 2.
6. Abraham Silberschatz, Peter B. Galvin and Greg Gagne, Operating System Concepts, Wiley.
7. William Stallings, Operating Systems: Internals and Design Principles, Pearson.